

Refactoring Report (Aufgabe 20)

Projekt: KlausurCraft

Datum: 01.04.2026

1) Pattern: Extract Function (Extract Method)

Katalog: <https://refactoring.com/catalog/extractMethod.html>

Stelle im Code: `HomeGenerateFlow.generateExamNow(...)` in `src/main/java/simon/klausurcraft/ui/h`

Die Methode bündelt mehrere Verantwortlichkeiten in einem Block: Auswahl geeigneter Subtasks, Verteilungsplanung, zufällige Variantenauswahl, Assembly-Erzeugung und Export-Trigger. Eine Aufteilung in kleine Funktionen wie `collectEligibleSubtasks(...)`, `chooseRandomVariant(...)` und `buildTaskAssembly(...)` würde die Lesbarkeit erhöhen und Teilverhalten separat testbar machen. Das Pattern ist hier sinnvoll, weil Fehlerfälle (z. B. keine feasible Kombination) klarer pro Teilphase behandelt werden können.

2) Pattern: Encapsulate Collection

Katalog: <https://refactoring.com/catalog/encapsulateCollection.html>

Stelle im Code: `Task.getSubtasks()` in `src/main/java/simon/klausurcraft/task/Task.java:28` und `Subtask.getVariants()` in `src/main/java/simon/klausurcraft/task/Subtask.java:45`.

Aktuell werden interne, mutable Listen direkt nach außen gegeben, wodurch Aufrufer den Objektzustand ohne zentrale Regeln verändern können. Mit Encapsulate Collection (z. B. `getSubtasksView()/getVariantsView()` als unveränderliche Sicht plus kontrollierte `add/remove`-Methoden) werden Invarianten besser geschützt. Das reduziert Seiteneffekte und macht spätere Validierung oder Autosave-Hooks einfacher integrierbar.

3) Pattern: Split Phase

Katalog: <https://refactoring.com/catalog/splitPhase.html>

Stelle im Code: `TaskXmlStore.load(...)` in `src/main/java/simon/klausurcraft/task/io/TaskXmlStore.java:94` und `parseTasks(...)` in `src/main/java/simon/klausurcraft/task/io/TaskXmlStore.java:94`.

Der Ladeprozess kombiniert aktuell Konfiguration/Validierung des XML-Parsers, Dateieinlesen und Mapping in Domänenobjekte in einem Ablauf. Mit Split Phase kann man den Ablauf explizit in getrennte Phasen zerlegen, z. B. `buildValidatedDocument(...)` und `mapDocumentToModel(...)`. Dadurch werden Fehlerursachen klarer zuordenbar, und jede Phase lässt sich isoliert testen und wiederverwenden.